

AUTONOMOUS AI SYSTEM

Continuous Coding Executive Agent

24/7 Autonomous AI That Builds While You Sleep



Self-Directed

Finds & executes work



Always On

PM2 continuous loop



Constitution-Governed

8 immutable limits

What We're Building



The Concept

An autonomous AI agent that runs **24/7 via PM2**, continuously monitoring a prioritized goal queue and executing work **without waiting for human prompts**.

PARADIGM SHIFT

Flips the traditional reactive model. Instead of "you prompt, it responds", it operates as **"a junior developer with a task board"**.



Core Mechanism

Spawns **Claude Agent SDK workers** to execute goals, validates results with **deterministic verifiers**, and immediately moves to the next task.



Work Hierarchy

1

Goals

What to build — directories in workspace/ with PROMPT.md

2

Steps

Auto-generated breakdown for complex goals (>100 turns)

3

Contracts

Scoped worker sessions with turn budgets



Goals flow through lifecycle: drafts/ → ondeck/ → in-progress/P{0-4}/ → completed/

THE CHALLENGE

Reactive AI Limitations

The Problem: Traditional agents cannot achieve true autonomy



Constant Human Intervention

Most AI coding agents are **reactive** — you must prompt, they respond. This creates a fundamental bottleneck.

- ✗ Interrupts flow state
- ✗ Prevents autonomy
- ✗ User must initiate



No Continuous Operation

Traditional agents **stop when you stop**. They cannot maintain momentum or work through a backlog.

- ✗ No overnight progress
- ✗ Queue management manual
- ✗ Single-turn execution



No Self-Directed Recovery

When failures occur, agents **wait for human diagnosis**. They cannot retry intelligently.

- ✗ No progress without human
- ✗ Single-strategy retries
- ✗ No auto-escalation



TRADITIONAL AGENT FLOW

Human Prompt



AI Response



Wait State



Cycle Repeats

0%

Autonomy

THE BREAKTHROUGH

Autonomous 24/7 Execution

The Solution: Continuous operation via PM2

AUTONOMOUS FLOW

- 1 Check Inbox (v2.0)**
Gmail fetch, parse actionable intents, queue goals
- 2 Health Check**
Auth validation, disk space, dependency checks
- 3 Select & Execute**
Auto-selects highest-priority unblocked goal (P0 → P4)
- 4 Validate & Update**
Run verifiers, update state, Notion, Discord
- 5 Diagnose or Escalate**
Cycles through strategies: simplify, research, subtasks

🔄 Loop Continues — Never stops, always working

KEY INNOVATIONS

- 🌙 **Smart Sleep/Resume**
Sleeps only when queue empty, immediately resumes on new work
- ⏴ **Priority Queue**
Auto-selects highest-priority unblocked goal from workspace
- ✂️ **Auto-Breakdown**
Complex goals (>100 turns) split into 2-5 manageable steps
- 💬 **Async Communication**
Human interaction via markdown files — no chat interface

24/7

Continuous Operation

via PM2 Process Management

Core Architecture: Three-Layer Design



Agentic

src/agentic/

AI decisions requiring LLM inference

- Work selection
- Strategy & diagnosis
- Prompt building
- Goal breakdown

 Uses LLM



Deterministic

src/deterministic/

Mechanical operations, no AI

- File I/O & state
- Health checks
- Notion reporting
- Credentials

 No LLM



Core

src/core/

Loop orchestration & types

- Executive loop
- Type definitions
- Logging system
- Event bus

 Foundation



Identity

src/identity/

Agent communication presence

- Gmail inbox check
- Discord webhooks
- Intent parsing
- Notification throttle

 New

v2.0

Two-Repository Design

continuous-agent/

The brain — agent infrastructure ONLY

- Executive loop, worker spawner
- Verifiers, constitution
- No application code

ai-sandbox/

The output — ALL worker outputs

- projects/ with git history
- Isolated directories
- Everything agent builds



Constitution Enforced

Workers **NEVER** write to agent codebase. Separation is absolute — enforced at the deterministic layer.

EXECUTION ENGINE

The Executive Loop: 8 Phases of Autonomy

0.5 Check Inbox v2.0

Gmail fetch, parse actionable intents, queue goals

2 Process Inputs

Read needs-you.md responses, ingest queue.md

3b Auto-Breakdown

Split complex goals (>100 turns) into 2-5 steps

5 Validate

Run verifiers on output_path (NOT agent codebase)

7 Agentic Diagnosis

After 3+ failures, LLM analyzes root cause

1 Health Check

Auth validation, disk space, dependency checks

3 Select Work

Priority-based selection (P0→P4), auto-promotion

4 Execute

Resolve pattern, spawn Agent SDK worker

6 Update State

Goal bundles, ledgers, Notion, Discord

8 Block/Escalate

After 10 failures, write to needs-you.md

LOOP CHARACTERISTICS

∞ Continuous

Never stops — always checking, always ready

⚡ Immediate Handoff

Picks next goal instantly after validation

🛌 Smart Sleep

Sleeps only when queue empty, resumes on new work

💓 Self-Healing

Retries with different strategies, escalates intelligently

Safety & Intelligence: Key Systems



The Constitution

Eight **immutable hard limits** in workspace/constitution.md that the agent cannot override. Enforced at the deterministic layer.

1. No spending beyond \$20/month per service
2. No permanent deletions (archive only)
3. No external publishing without approval
4. No credential exposure
5. No access control expansion
6. No output in agent codebase
7. All activity must be logged
8. 10 retries minimum before blocking



Human Interaction

Communicates asynchronously via `workspace/needs-you.md`

When blocked, agent writes:

Action

Why Can't Do

Response

[APPROVED] Proceed

[DECISION] Make choice

[INFO] Provide info

[SKIP] Skip goal

🕒 Agent detects responses within ~30 seconds



Retry System with Strategy Rotation

When a worker fails, the agent doesn't just retry — it selects a **different strategy** each time.



Simplify Scope



Research First



Break into Subtasks



Different Tools

Evolution: From v1.0 to v2.1

1.0

Jan 24, 2026

- 8-phase executive loop
- Two-repo architecture
- Worker delegation
- Constitution (8 limits)
- Contract system
- Verifier system
- Capability tracking
- PM2 deployment

Foundation

1.1

Jan 25, 2026

- Incremental execution
- Self-enhancement
- Prompt management
- Intent classification
- Strategy rotation
- Agentic diagnosis
- Self-improvement triggers
- needs-you.md interaction

Intelligence

1.2

Jan 28, 2026

- Goal bundles
- Workspace lifecycle
- STEPS.json tracking
- CONTRACTS.jsonl
- PROGRESS_LOG.md
- Notion reporting
- Goal scanner
- Project registry

Organization

1.3

Feb 4, 2026

- Three-tier credentials
- Physical separation
- Leak detection
- Tier 3 format helpers
- OAuth-first auth
- No API key needed

Security

2.0

Mar 29, 2026

- Gmail inbox checking
- Discord notifications
- Execution patterns (4)
- Skills & playbooks
- Track record system
- Dashboard UI
- Pipeline executor
- Feature flags

Identity

2.1

LATEST

Apr 1, 2026

- Vendor abstraction layer
- Claude, Codex, Kimi
- Per-goal vendor override
- Kimi dual-mode support
- Vendor E2E tests
- GitHub Pages deployment
- Multi-vendor comparison

Multi-Vendor

65+

Days

5

Releases

40+

Features

Development Velocity

From basic loop to full identity system with dashboard and multi-pattern execution

CURRENT CAPABILITIES

v2.0 Capabilities: Agent Identity & Execution Patterns



Agent Identity System

The agent now has its own **communication presence** — it can receive goals via email and push notifications to Discord. All opt-in, disabled by default.



Gmail Integration

OAuth2 flow, fetch/send/archive, intent parsing from unread emails

IDENTITY_ENABLED



Discord Webhooks

Completion/blocked notifications with throttling, rich embeds

DISCORD_ENABLED



Skills & Playbooks

Reusable knowledge libraries with **execution history tracking**



Skills

Atomic tool/API how-tos



Playbooks

Goal-oriented workflows



Track Record

Success rate, confidence scoring



Execution Patterns

Goals can specify execution patterns in PROMPT.md frontmatter. Resolved via:
PROMPT.md → **playbook** → **system default**

plan-then-execute

Default — research then build

DEFAULT

loop-until-progress

Keep iterating while making gains

plan-mode

Read-only analysis, no code changes

deterministic-pipeline

Fixed multi-step sequence



Dashboard UI

Next.js/React read-only dashboard for visualizing agent state in real-time



Goal pipeline view



Active worker status



Needs-you queue



Activity feed

V2.1 ACHIEVEMENT

Multi-Vendor Worker Support

Vendor abstraction layer with three backends



Claude

Default

Backend: Claude Agent SDK

Auth: OAuth token

Mode: Full agentic execution

88

Score /130

3rd place



OpenAI Codex

CLI-based

Backend: Codex SDK threads

Auth: codex login

Mode: Different interaction patterns

115

Score /130

🏆 1st place



Kimi K2.5

Dual-mode

Backend: Wire SDK + CLI

Auth: kimi login

Mode: KIMI_MODE=wireCli

92

CLI
2nd

87

Wire
4th



Multi-Vendor Comparison

Same prompt ("build a finance dashboard") executed across all 4 vendor modes

Live deployment at:

jackzhaojin.github.io/ai-sandbox



Key Features

- ✓ Per-goal vendor override via PROMPT.md frontmatter
- ✓ All outputs normalized to AgentWorkerMessage format
- ✓ E2E vendor tests for all backends
- ✓ GitHub Pages CI deployment pipeline

Cloud Database Migration

Migrate ledgers & AI state from local files to cloud database



The Vision

Move all **mutable agent operational data** out of local flat files and into a cloud database (Supabase or Cosmos DB). Local files become optional cache/fallback — the cloud DB becomes the source of truth.

UNLOCKS

🔍 Queryability

👥 Concurrent access

🛡️ Durability

📈 Real-time observability



Database Provider

★ Supabase

Recommended — credentials already exist in .env.app

☁️ Cosmos DB

Alternative — if Azure integration preferred



What Moves to Cloud DB



Work ledger

work_ledger



Executive logs

executive_logs



Contract events

contract_events



Step tracking

goal_steps



Evolution log

evolution_log



Retrospectives

retrospectives



Agent state

agent_state



Project registry

project_registry

STAYS LOCAL

- constitution.md (immutable, version-controlled)
- PROMPT.md files (human-authored, git-tracked)
- .env.* files (credentials, never in DB)
- capabilities/*.yaml (skill definitions)

Problems Solved by Cloud Database



No Queryability

Problem: Cannot aggregate, filter, or analyze historical data without parsing JSONL files line by line

SOLUTION

SQL queries, indexes, and real-time analytics on operational data



No Scalability

Problem: As ledgers grow, file I/O becomes a bottleneck

SOLUTION

Database indexing, query optimization, horizontal scaling



No Durability

Problem: Local files are lost if the machine is wiped or disk fails

SOLUTION

Cloud-backed persistence with automatic backups and replication



Limited Observability

Problem: No dashboards or real-time queries against operational data

SOLUTION

Live dashboards, real-time metrics, operational intelligence



No Concurrent Access

Problem: Multiple processes reading/writing JSONL files risk corruption

SOLUTION

ACID transactions, row-level locking, safe multi-process access

STORAGE PROVIDER ABSTRACTION

```
interface StorageProvider {  
  appendLedgerEntry(entry): Promise  
  queryLedger(filter): Promise  
  readSteps(goalSlug): Promise  
  writeSteps(goalSlug, steps): Promise  
  getState(key): Promise  
  setState(key, value): Promise  
}
```

● LocalFileProvider ● CloudDbProvider

Technical Deep Dive



Three-Tier Credential System

Credentials are **physically separated** into three files to prevent accidental leakage. Leak detection validates no cross-tier contamination.

Tier 1: Executive — .env.executive
Loop config, Notion API, identity settings

Tier 2: Worker — .env.worker
Claude SDK auth (OAuth token)

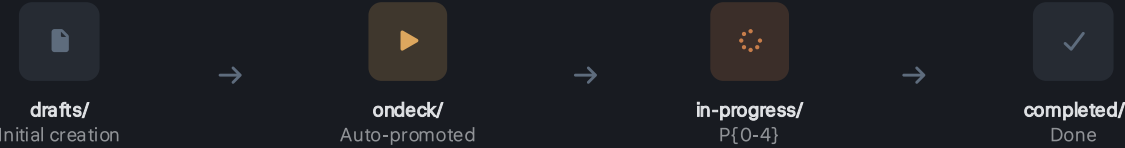
Tier 3: Application — .env.app
App credentials (DB, storage) with APP_ prefix

SECURITY
Tier 1 keys never reach workers. Spawner validates on every execution.



Goal Bundle Lifecycle

Goals flow through a structured lifecycle with **automatic promotion by priority**.



Per-Bundle Files


PROMPT.md
Goal definition with YAML frontmatter


STEPS.json
Machine-readable step tracking (source of truth)


PROGRESS_LOG.md
Append-only human-readable timeline


CONTRACTS.jsonl
Per-bundle contract event log



Append-Only Ledgers

- ✓ work-ledger.jsonl: All contract events
- ✓ capability-ledger: Skill confidence scores
- ✓ executive-*.log: Daily executive logs
- ✓ worker-*.log: Per-contract worker logs

RULE
Never truncate or modify existing entries



Special Workflows

LOOKING AHEAD

Roadmap Forward: Beyond v2.2

2.2

Current: Cloud Database Migration

Foundation for future capabilities

v2.2 is **foundational infrastructure** that unblocks better observability, reliability, and multi-machine operation. Prioritized before integration wiring to tighten the codebase first.



Real-Time Subscriptions

Supabase supports real-time — could enable **live dashboard updates** without polling

🕒 Out of scope for v2.2, worth noting for v2.4



Retention Policies

Should old ledger entries be **archived/purged** after N days?

∞ Keep all for now, add retention later



Multi-User / Second Agent

Configure separate auth tokens, verify **isolation** between agents

⚠️ **High risk — not yet designed**



Enhanced Observability

Queryable operational data for **analytics and insights**

☰ Depends on v2.2 cloud migration



Integration Wiring

Connecting cloud state to executive loop — **dashboard writer in loop**, live rendering, Gmail/Slack API live testing

An AI That Doesn't Wait for Instructions

It finds work, executes with intelligence, learns from failures, and only asks for help when truly stuck. The Continuous Executive Agent represents a **fundamental shift** from reactive to proactive AI.



Self-Directed

Monitors queue, selects priorities, executes without prompting



Self-Healing

Retries with different strategies, breaks down complex problems



Constitution-Governed

Operates within immutable safety limits, never exceeds boundaries